

Stack Using Array

Aim:

To write a program to implement a stack using an array and perform basic operations.

Algorithm:

Step 1: Start Begin program execution.

Step 2: Initialize Stack – Define a stack of size MAX (5).

Step 3: Initialize top – Set top = -1 to indicate the stack is empty.

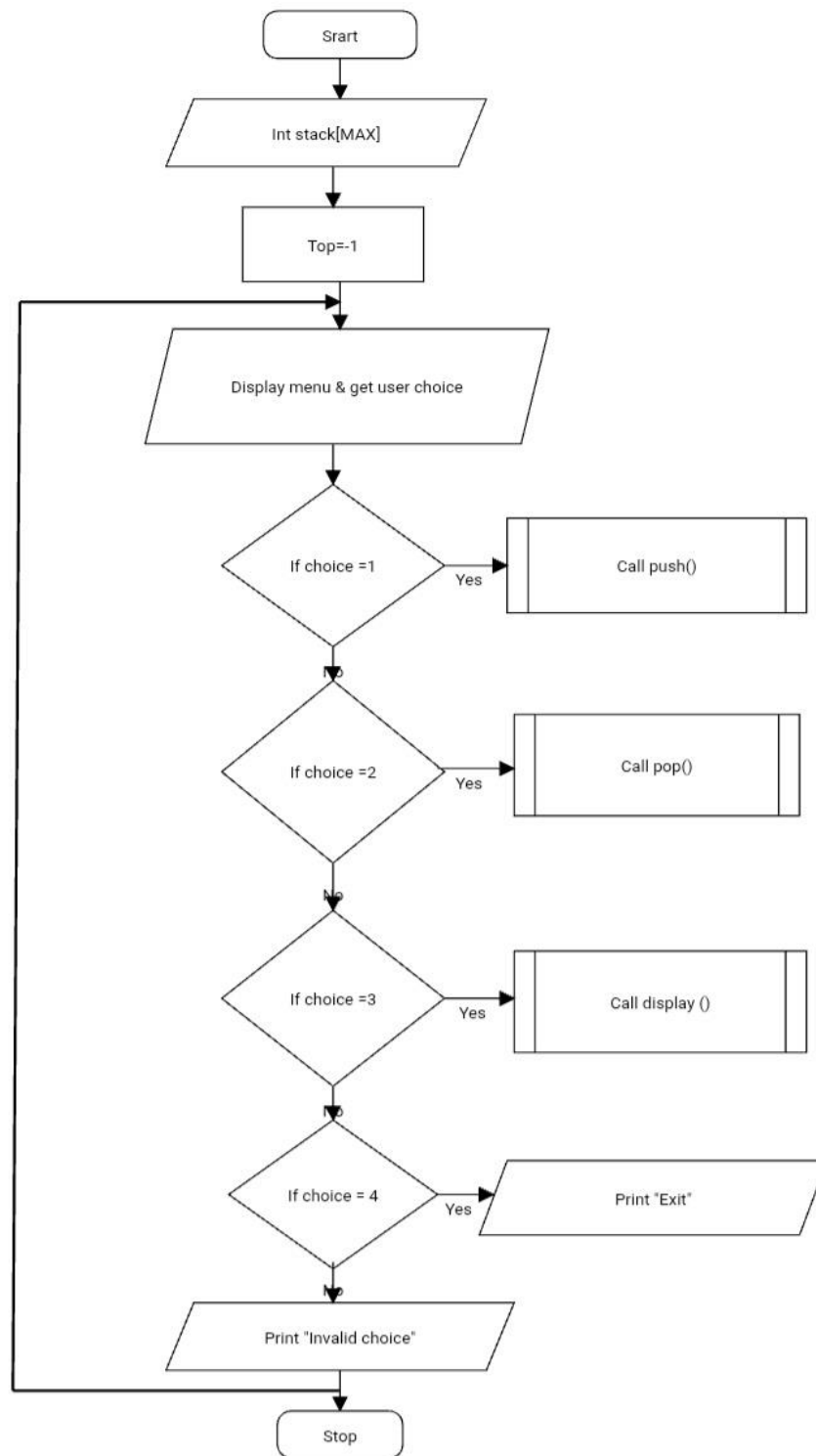
Step 4: Display Main Menu – Show options: 1. Push 2. Pop 3. Display 4. Exit.

Step 5: Read User Choice – Read the choice from the user.

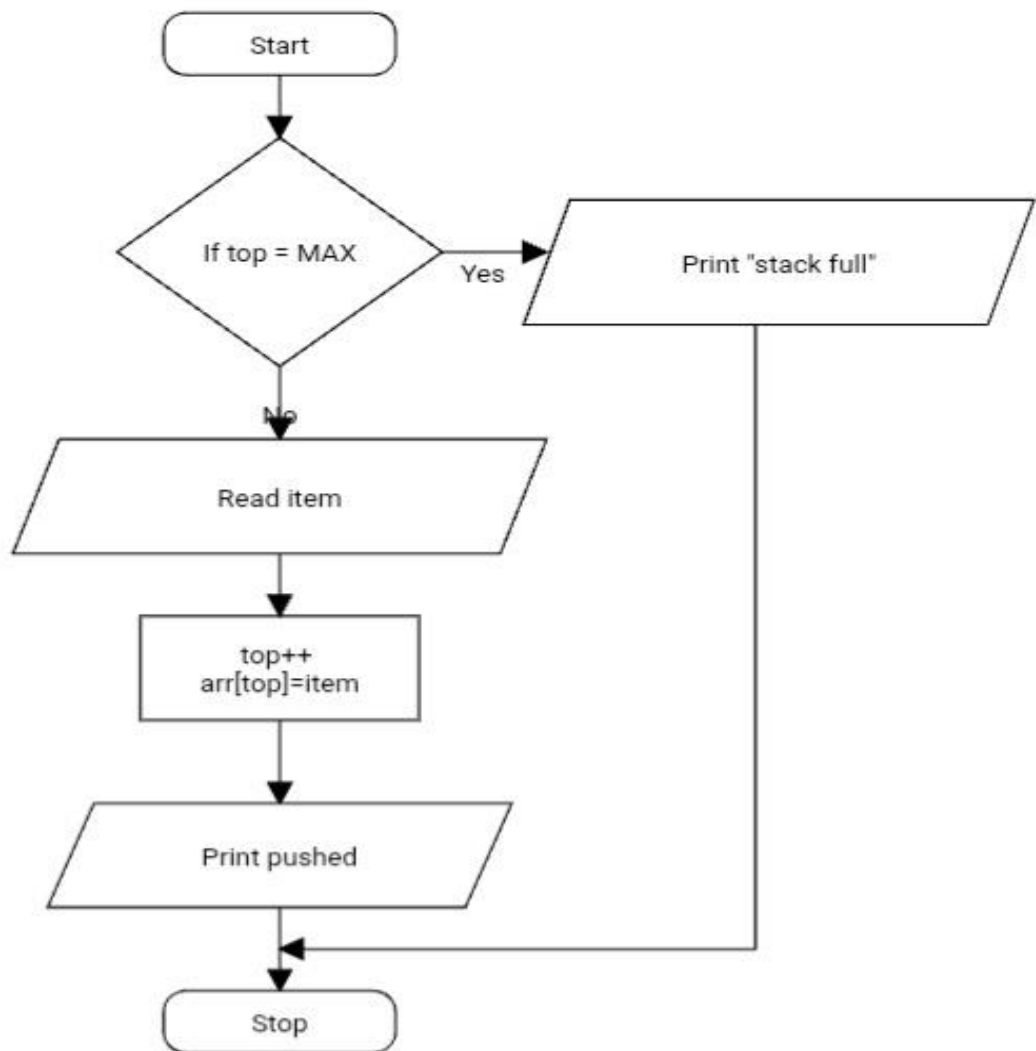
- If choice = 1, go to Push Operation Steps.
- If choice = 2, go to Pop Operation Steps.
- If choice = 3, go to Display Operation Steps.
- If choice = 4, go to Exit Operation Steps.
- Otherwise, print "Invalid choice" and return to Step 4 (Display Main Menu).
 - Push () Operation Steps (Choice = 1)
 - Step P1: Check Stack Overflow – If top == MAX - 1, print "Stack Overflow" and return to Step 4 (Display Main Menu).

- Step P2: Read Item – Read the element to push from the user.
- Step P3: Increment top – $\text{top} = \text{top} + 1$.
- Step P4: Assign Stack Element – $\text{stack}[\text{top}] = \text{item}$.
- Step P5: Print Success Message – Print "Element pushed successfully" and return to Step 4 (Display Main Menu).
- Pop() Operation Steps (Choice = 2)
 - Step O1: Check Stack Underflow – If $\text{top} == -1$, print "Stack Underflow" and return to Step 4 (Display Main Menu).
 - Step O2: Print Popped Element – Print $\text{stack}[\text{top}]$.
 - Step O3: Decrement top – $\text{top} = \text{top} - 1$ and return to Step 4 (Display Main Menu).
- Display() Operation Steps (Choice = 3)
 - Step D1: Check if Stack is Empty – If $\text{top} == -1$, print "Stack is empty" and return to Step 4 (Display Main Menu).
 - Step D2: Loop Through Stack – For $i = \text{top}$ down to 0, print $\text{stack}[i]$.
 - Step D3: Return to Main Menu – Go back to Step 4 (Display Main Menu).
- Exit() Operation Steps (Choice = 4)
 - Step E1: Print Exit Message – Print "Exiting program".
 - Step E2: End – Program execution

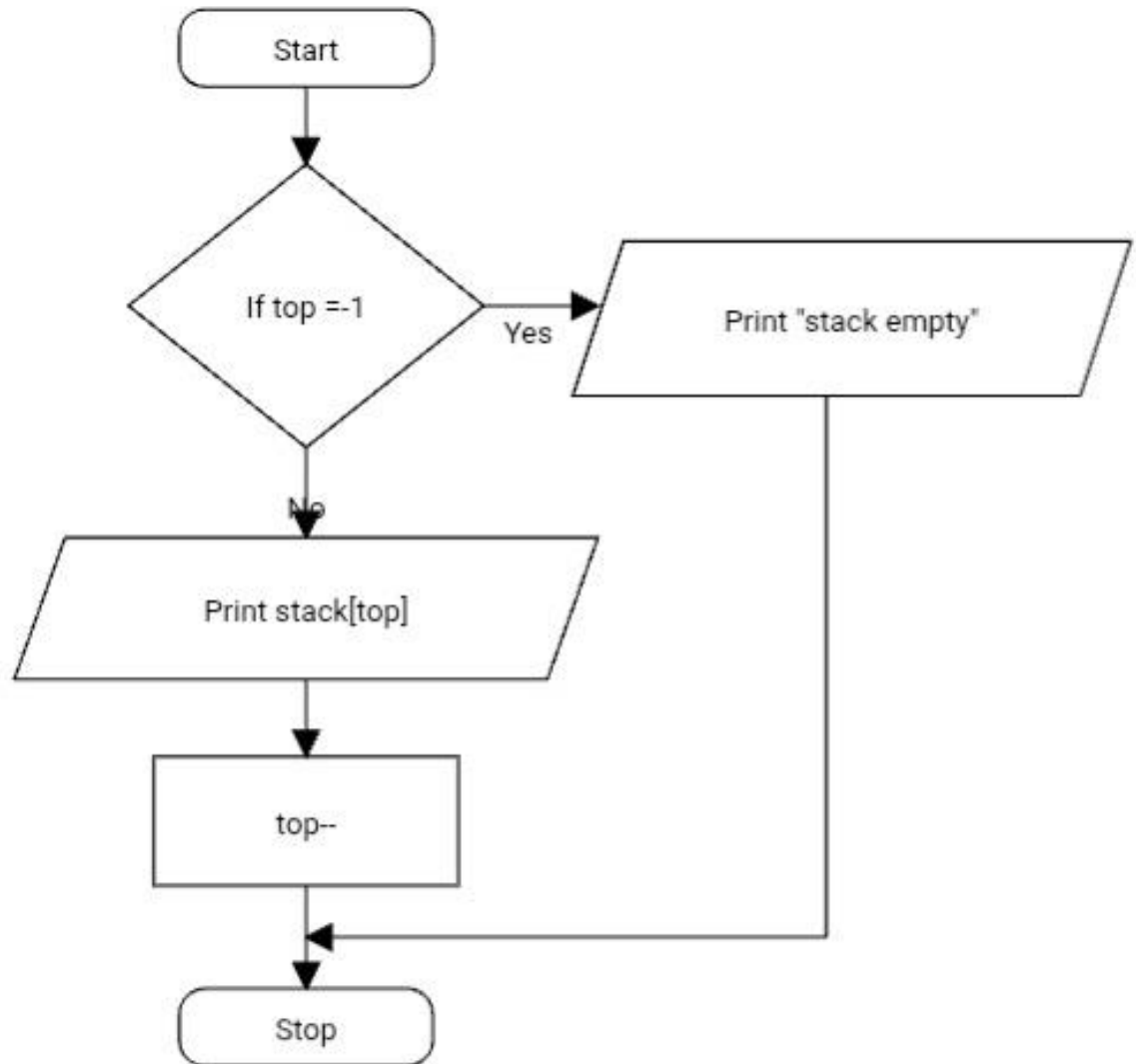
Flowchart:



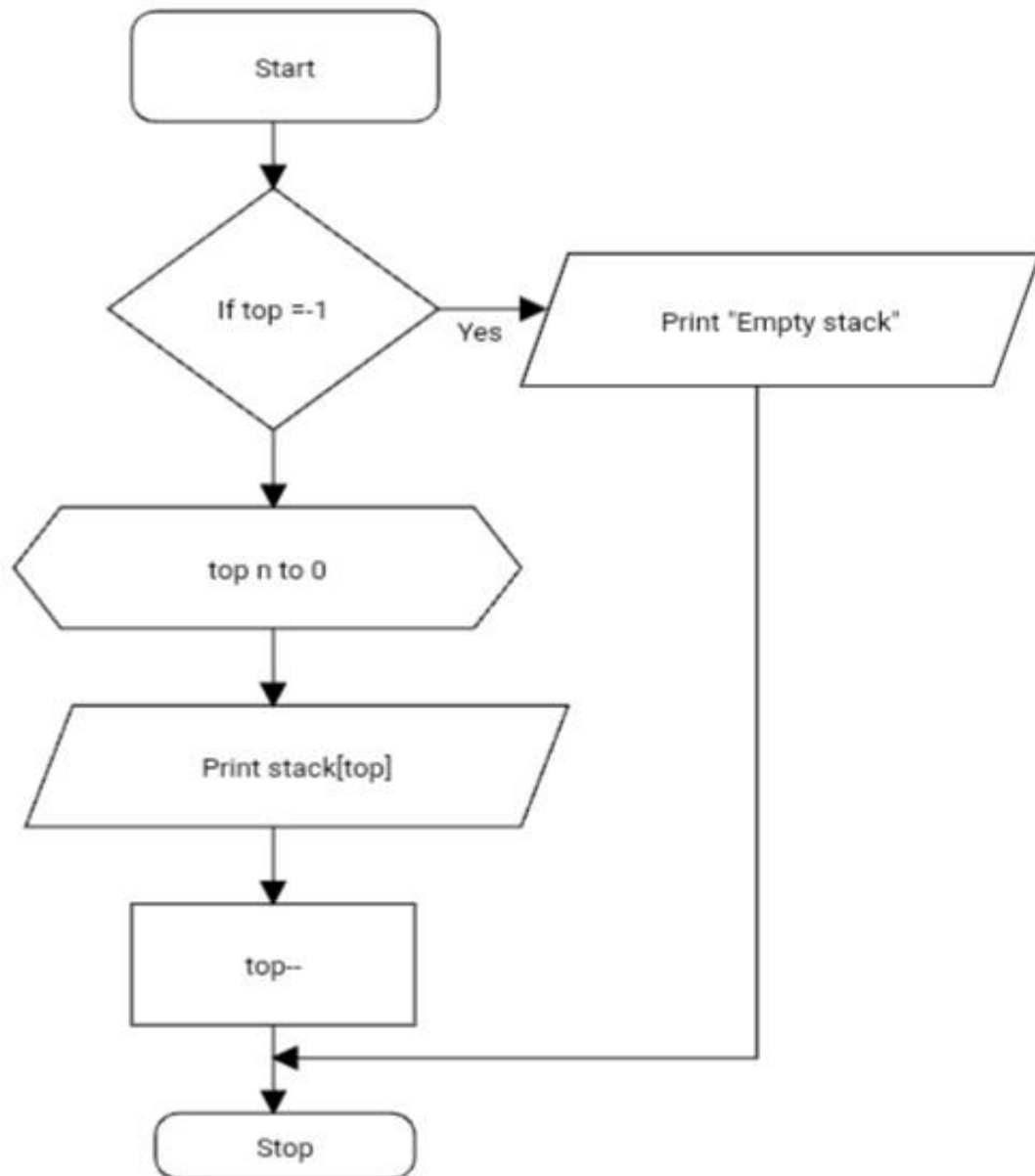
1.push()



2.pop()



3.Display()



Source code:

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// Push operation
```

```
void push()
```

```
{
```

```
    int item;
```

```
    if (top == MAX - 1)
```

```
    {
```

```
        Printf("Stack Overflow!\n");
```

```
        return;
```

```
    }
```

```
    Printf("Enter element to push: ");
```

```
    Scanf("%d", &item);
```

```
    Stack[++top] = item;
```

```
    Printf("Element pushed successfully.\n");  
}  
  
// Pop operation  
void pop()  
{  
    if (top == -1)  
    {  
        Printf("Stack Underflow!\n");  
        return;  
    }  
    Printf("Popped element: %d\n", stack[top--]);  
}  
  
// Display stack elements  
void display()  
{  
    if (top == -1)  
    {
```



```
    Printf("Stack is empty.\n");

    return;

}

Printf("Stack elements:\n");

for (int i =choice;i>= 0; i--)

    Printf("%d\n", stack[i]);

}

int main()
{

    int choice;

    do

    {

        Printf("\n--- Stack Menu ---\n");

        Printf("1. Push\n2. Pop\n3. Display\n4. Exit\n");

        Printf("Enter your choice: ");

        Scanf("%d", &choice);

        switch (choice)

        {
```

case 1:

push();
break;

case 2:

pop();
break;

case 3:

display();
break;

case 4:

printf("Exiting program.\n");
break;

default:

printf("Invalid choice!\n");

}

} while (choice != 4);

return 0;

}

OUTPUT:

--- Stack Menu ---

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 12

Element pushed successfully.

--- Stack Menu ---

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 13

Element pushed successfully.

--- Stack Menu ---

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack elements:

13

12

--- Stack Menu ---

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 2

Popped element: 13

--- Stack Menu ---

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 3

Stack elements: 12

--- Stack Menu ---

1. Push

2. Pop

3. Display

4. Exit

Enter your choice:

4

Exiting program.

Result:

Thus the program is complied and verified successfully with verified output.